AI-Powered Workforce Operations Analytics Assistant

An analytics application built for **Operations Managers / Workforce Managers** — not data scientists. It combines a live, filterable operations dashboard with a natural-language query assistant, so a manager can monitor workforce health at a glance and then dig into specifics by asking plain-English questions instead of writing SQL.

Problem It Solves

Operations managers need to answer questions like *"which shift has the worst absenteeism?"* or *"is overtime creeping up at a particular facility?"* every day, but usually that means asking a data or IT team to write a query. This tool puts a live dashboard and a natural-language query assistant directly in the manager's hands.

Business Use Case

A manager opens the app and immediately sees:

1. **Dashboard** — headline KPIs (headcount, attendance, absenteeism, overtime, attrition, efficiency).
2. **Monitor** — an operational alerts panel that flags high overtime, high absenteeism, low efficiency, or high error rates using simple, transparent thresholds (fully adjustable in the sidebar).
3. **Ask** — a natural-language box to investigate anything not already covered by the dashboard.
4. **Analyze** — the assistant returns the SQL it generated, the result table, and a one- or two-line business insight.
5. **Decide** — armed with the numbers, the manager can act (rebalance shifts, address a facility's absenteeism, investigate a department's error rate, etc.).

Features

- **Operational dashboard** with KPIs for workforce, attendance, productivity, overtime/utilization, and leave — computed live from the database, not hard-coded.
- **Filters**: date range, facility, department, shift, employment type. Every KPI and chart reacts to the selected filters.
- **Charts** (Plotly): attendance/absenteeism trend, efficiency by department, absenteeism by shift and facility, workforce distribution, overtime by department/facility and over time.
- **Operational alerts**: transparent, configurable business rules (e.g. "overtime > 3 hrs/day", "efficiency < 80%") — no black-box machine learning.
- **Natural-language query assistant**: ask a workforce question in plain English; Gemini translates it into SQL, the app runs it, and shows the question, generated SQL, result table, and a short business insight.

- **Basic SQL safety validation**: only single `SELECT` statements are executed. `INSERT`, `UPDATE`, `DELETE`, `DROP`, `ALTER`, and similar destructive keywords are rejected before anything touches the database.
- **Simple analytical drill-down** for facility investigation (by shift, department, employment type), instead of an automated root-cause-analysis engine.

Architecture

```
Streamlit (app.py)
    ↓
Gemini (sql_generator.py) → SQL
    ↓
SQLite (workforce.db, accessed via database.py)
    ↓
Analytics (analytics.py, pandas-based KPI & chart calculations)
    ↓
Dashboard + Business Insight (rendered back in Streamlit)
```

- `app.py` — Streamlit UI: sidebar filters, dashboard sections, alerts, and the NL assistant.
- `database.py` — SQLite connection handling, SQL safety validation, and typed query helpers.
- `analytics.py` — All KPI/business-logic calculations and chart-ready aggregations (pandas).
- `sql_generator.py` — Gemini prompt/schema context, SQL generation, and business-insight generation.
- `generate_data.py` — Synthetic dataset generator; regenerate `workforce.db` any time.

Dataset

The bundled `workforce.db` is a **synthetic, reproducible** dataset (no real personal data), generated by `generate_data.py`, with deliberate but realistic operational patterns:

- **~650 employees** across **6 facilities**, **10 departments**, **3 shifts** (Morning/Evening/Night), and **3 employment types** (Permanent/Contract/Temporary).
- **~9 months of attendance history** (~100K+ attendance records).
- **Multiple performance records per employee** over time (~67K+ records).
- **~680 leave requests** spread across leave types.
- Built-in correlations: certain facilities run under more staffing pressure (higher overtime and absenteeism), Night shift trends worse on absenteeism/overtime than Morning, and certain departments (Packaging, Dispatch) trend lower on efficiency — without being so uniform that the data looks artificially constructed.

Regenerate anytime with:

```
python generate_data.py
```

Business Logic Definitions (used consistently across the dashboard and NL assistant)

- **Attendance Rate** = Present days ÷ total logged days.
- **Absenteeism Rate** = Absent days ÷ total logged days.
- **Overtime** = `SUM(overtime_hours)` from `ATTENDANCE_LOGS` (a per-day field, so no double-counting).
- **Efficiency %** = mean of `PERFORMANCE_METRICS.efficiency_pct` (already normalized units/target per record).
- **Attrition Rate** = employees who exited within the selected window ÷ average active headcount over that same window (not a raw termination count).

Tech Stack

- **Frontend/UI:** Streamlit
- **Language:** Python
- **LLM:** Google Gemini, via the current Google Gen AI SDK (https://ai.google.dev/gemini-api/docs/libraries) (`google-genai`)
- **Database:** SQLite
- **Analytics:** Pandas
- **Visualization:** Plotly

Setup Instructions

Prerequisites

- Python 3.10+
- A Google Gemini API key (get one free (https://aistudio.google.com/apikey))

Install & Run

```
pip install -r requirements.txt
```

Copy `.env.example` to `.env` and set your key:

```
GOOGLE_API_KEY=your_key_here
```

The database (`workforce.db`) is already included and ready to use. If you want to regenerate it:

```
python generate_data.py
```

Then start the app:

```
streamlit run app.py
```

The dashboard works immediately without an API key. The **Ask the Workforce Assistant** section requires a valid `GOOGLE_API_KEY` to generate SQL from natural language.

Example Questions

- Which facility has the highest absenteeism?
- Which department has the highest overtime?
- Show me employees who worked more than 3 hours of overtime this month.
- Compare productivity between morning and night shifts.
- Which department has the lowest average efficiency?
- How many contract employees are currently active?
- Which facility processed the highest number of units?
- Show absenteeism trends for the last 3 months.
- Which employees have high overtime and low efficiency?
- What is the attrition rate by facility?
- Which shift has the highest absenteeism?
- List contract workers with overtime greater than 2 hours.

Project Structure

```
AI-Powered-Workforce-Operations-Analytics-Assistant/
│
├── app.py              # Streamlit UI: dashboard, filters, alerts, NL assistant
├── database.py          # SQLite access + SQL safety validation
├── analytics.py         # KPI calculations & chart aggregations (pandas)
├── sql_generator.py     # Gemini schema/prompt, SQL generation, insight generation
├── generate_data.py     # Synthetic dataset generator
│
├── workforce.db          # Pre-generated SQLite database (ready to use)
│
├── requirements.txt
├── .env.example
└── README.md
```

What This Project Intentionally Does NOT Include

Per design scope, this stays a practical, explainable analytics tool rather than an ML/AI platform: no forecasting, no attrition-prediction models, no RAG/vector databases, no multi-agent orchestration, and no infrastructure beyond SQLite + Streamlit. Operational alerts use transparent, adjustable thresholds rather than machine learning.